

PROFESSIONAL TRAINING MATERIAL · 2026 EDITION

DevOps & Cloud – 90-Day Learning Journey

DAY 02 · MODULE 2

Foundation of Linux & Operating Systems

Kernel, Shell, File System, Commands, Permissions, SSH & More

9 Topics

Bullet-First

Try-It Tasks

Mini Quizzes



[B] Beginner-Intermediate

[T] ~90 min read

Aryashree Pritikrishna

<https://github.com/aryashreep/learn-devops-in-90-days>

[*] From Zero to Production – Session by Session [*]

Table of Contents — Day 02: Linux for DevOps

Ch 01	Foundations of Linux and Operating Systems	[B] Beginner [T]8m
Ch 02	Virtualization and Linux VMs	[B] Beginner [T]7m
Ch 03	Exploring the Linux File System	[B] Beginner [T]8m
Ch 04	Managing Software on Linux	[B] Beginner [T]7m
Ch 05	Working with Text Editors (Nano & Vim)	[B] Beginner [T]6m
Ch 06	Linux User and Permission Management	[I] Intermediate [T]9m
Ch 07	Mastering the Command Line	[I] Intermediate [T]8m
Ch 08	Introduction to Shell Scripting	[I] Intermediate [T]9m
Ch 09	Environment Variables & Networking Essentials + SSH	[I] Intermediate [T]10m

CHAPTER 01

[B] Beginner

[T] 8 min read

1/9

Foundations of Linux and Operating Systems

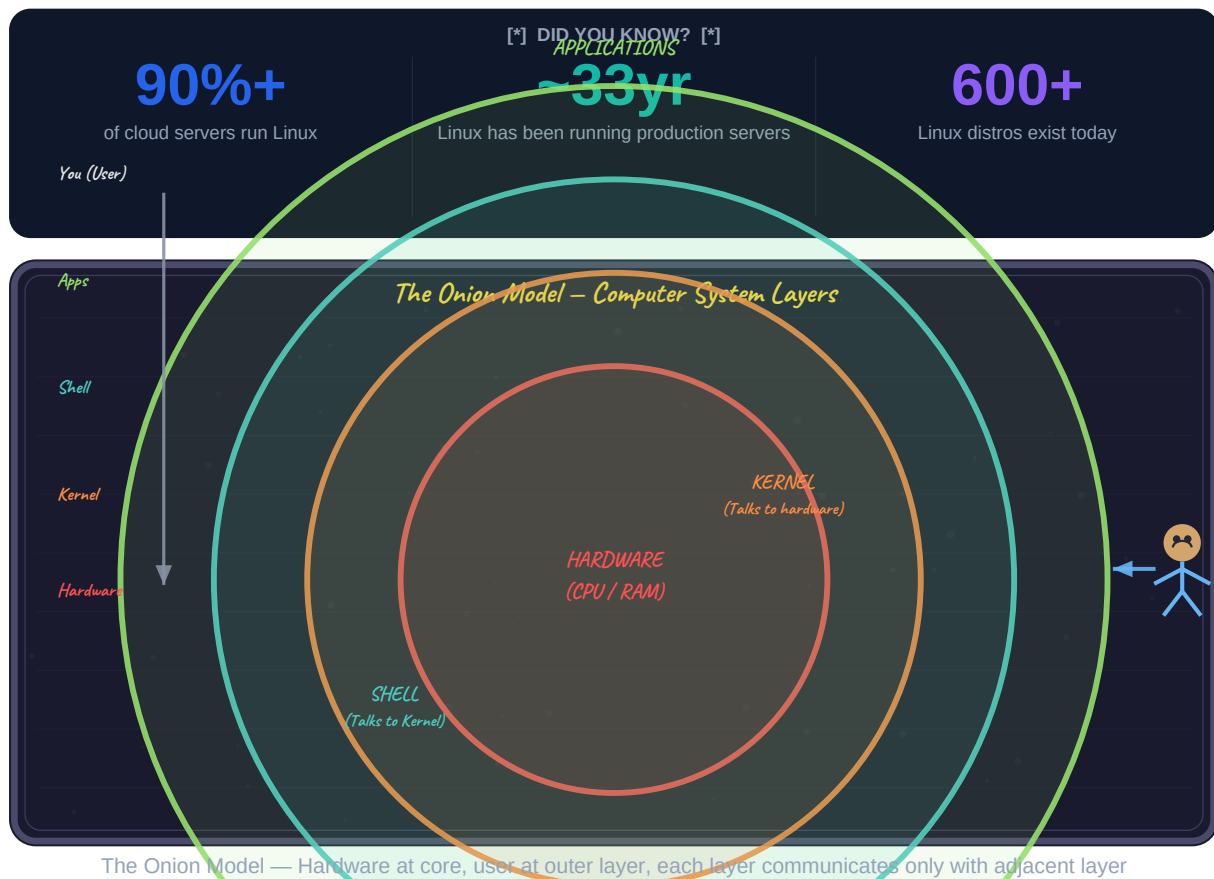
[TARGET] LEARNING OBJECTIVES

By the end of this chapter you will be able to:

- [*] Explain what an Operating System (OS) does in plain English using the Restaurant analogy
- [*] Define Kernel and Shell and describe the difference between them
- [*] Identify why Linux dominates cloud infrastructure (AWS, Azure, GCP)
- [*] Use `uname -r` and `cat /etc/os-release` to inspect a Linux system

[IDEA] ANALOGY

Computer = Restaurant: Hardware is the Kitchen, Applications are Customers, OS is the Restaurant Manager who coordinates everything without chaos



What is an Operating System?

- **OS definition:** software layer between hardware and applications
- **Without OS:** every app would need to directly control CPU, RAM, disk — impossible chaos
- **OS responsibilities:** memory management, process scheduling, file I/O, security, networking
- **Linux:** free, open-source OS kernel created by Linus Torvalds in 1991

- **Why Linux for DevOps:** AWS, Azure, GCP — all run Linux on their servers

Kernel vs Shell — The Two Core Concepts

- **Kernel:** the core — only part that talks directly to hardware
 - → manages CPU, RAM, devices at the lowest level
 - → you NEVER interact with it directly
 - → runs in privileged "kernel space" — protected from user mistakes
- **Shell:** the user interface — interprets your typed commands
 - → sends instructions to kernel on your behalf
 - → common shells: bash (default), zsh (macOS), sh (scripting)
 - → when you type `ls -la`, Shell interprets it, Kernel executes it
- **Distributions (Distros):** Linux kernel + bundled tools
 - → Ubuntu — best for beginners, DevOps, cloud (AWS default)
 - → CentOS / RHEL — enterprise servers, strict stability
 - → Debian — ultra-stable, basis for Ubuntu
 - → Alpine — tiny (5MB), used inside Docker containers

[X] Without OS	[OK] With Linux OS
• Each app manually controls hardware • No memory isolation between programs • System crashes = everything lost • Manual process for every operation	• OS manages all hardware access centrally • Each process isolated in protected memory • Kernel handles crash recovery • Shell scripts automate repetitive operations

Distro	Based On	Best For	Key Fact
Ubuntu	Debian	DevOps, Cloud, Beginners	Default on AWS EC2 free tier
CentOS/RHEL	Red Hat	Enterprise servers, production	yum/dnf package manager
Alpine	Independent	Docker containers	Only 5MB — smallest Linux
Debian	Independent	Stable servers, scripting	Basis for Ubuntu
Kali Linux	Debian	Security testing, pen testing	Pre-installed hacking tools

[BUILDING] REAL-WORLD EXAMPLE**Netflix****Situation:**

- Needed to serve 260M+ users globally — required OS that scales to thousands of servers automatically

What they did:

- Standardised entirely on Linux (Ubuntu) across all production servers
- Used Linux cgroups and namespaces (kernel features) for container isolation
- Automated server management via Linux shell scripts + Terraform

Result:

- Zero Windows servers in production — 100% Linux across entire infrastructure
- Linux kernel features enabled Docker containers that power their microservices
- Cost savings: Linux is free vs Windows Server licensing fees

[!] COMMON MISCONCEPTIONS**[X] Myth:**

"Linux" is one operating system like Windows

[OK] Reality:

Linux is a KERNEL. Distributions (Ubuntu, CentOS, Alpine) are complete OSes built around it. "Linux" refers to all of them collectively

[X] Myth:

"GUI-less Linux" means it is broken or incomplete

[OK] Reality:

Production servers intentionally have NO graphical interface. CLI-only servers are faster, use less RAM, and are more secure — this is the professional standard

Key Commands — Try These Now

- **uname -r:** prints the Linux kernel version currently running on your system
- **cat /etc/os-release:** shows which Linux distro and version you are running
- **uname -a:** full system information — kernel, hostname, architecture
- **whoami:** prints your current username
- **hostname:** prints the computer name (hostname)

[>] TRY IT YOURSELF

1. Run `uname -r` in your terminal — write down the kernel version. Run `cat /etc/os-release` to see your distro
2. Install Ubuntu via VirtualBox (free) or WSL2 on Windows: search "Enable WSL2 Windows 11" in your browser
3. Create a free AWS account → launch a t2.micro EC2 instance → note it boots Ubuntu Linux in 90 seconds

[OK] QUICK SUMMARY

- **OS:** software layer between hardware and applications — coordinates everything
- **Kernel:** core of the OS — only component that directly controls hardware
- **Shell:** user interface to the OS — interprets your typed commands for the kernel
- **Linux:** free, open-source OS kernel — runs 90%+ of the world's cloud servers
- **Distros:** Ubuntu (DevOps/cloud), CentOS (enterprise), Alpine (containers)
- **uname -r** checks kernel version; **cat /etc/os-release** shows your distro

[?] MINI QUIZ — Test Your Understanding

Q1. What is the KERNEL in an operating system?

- a) The graphical desktop interface
- b) The core component that directly controls hardware
- c) The terminal or command prompt
- d) The file manager application

[OK] Answer: b — The Kernel is the core — it manages CPU, RAM, and hardware. Only the Kernel talks directly to hardware

Q2. A DevOps engineer needs to run 5 isolated test environments on ONE physical server cheaply. Best solution?

- a) Buy 5 separate physical servers
- b) Use 5 separate hard drives
- c) Use Linux virtualization to run 5 VMs on one machine
- d) Use 5 different user accounts

[OK] Answer: c — Virtualization runs multiple VMs on one physical machine — the core cost-efficiency benefit

Q3. Which Linux command shows which distro and version you are running?

- a) `uname -r`
- b) `ls -la`
- c) `cat /etc/os-release`
- d) `pwd`

[OK] Answer: c — cat /etc/os-release reads the OS release file — shows distro name, version, and ID

CHAPTER 02

[B] Beginner

[T] 7 min read

2/9

Virtualization and Linux VMs

[TARGET] LEARNING OBJECTIVES

By the end of this chapter you will be able to:

- [*] Explain virtualization using the "house divided into apartments" analogy
- [*] Distinguish Type 1 (Bare Metal) vs Type 2 (Hosted) Hypervisors with examples
- [*] Describe why VMs prevent "it works on my machine" errors
- [*] Set up a Linux VM using VirtualBox with an Ubuntu ISO

[IDEA] ANALOGY

Virtualization = magic wall dividing one large house into separate mini-apartments — each VM is an apartment with its own door, furniture, residents, and fire safety

[*] DID YOU KNOW? [*]

5 VMs

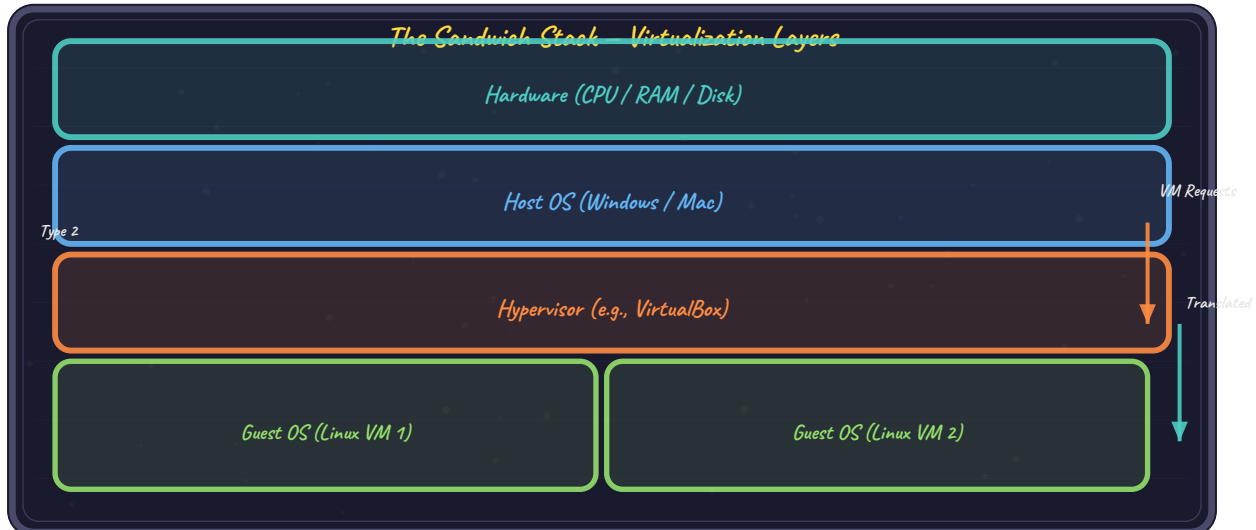
run on 1 physical machine — saves 80% hardware cost

90sec

to create a VM on AWS vs 6 weeks for physical

Isolated

crash one VM — rest of system stays safe



The Sandwich Stack — Guest OS runs on Hypervisor which translates requests to Host OS hardware

Key Concepts

- **Virtualization:** dividing one physical computer into multiple isolated environments
 - → each environment (VM) has its own virtual CPU, RAM, disk, network
 - → VMs are completely isolated — what happens in VM1 cannot affect VM2
- **Hypervisor:** the software that creates and manages VMs

→ → Type 1 (Bare Metal): installed directly on hardware — used in data centres (VMware ESXi, AWS Xen, KVM)

→ → Type 2 (Hosted): installed as an app on your existing OS — used for learning (VirtualBox, VMware Workstation)

→ → Type 1 is faster (no host OS overhead); Type 2 is easier to install

- **Host OS:** the real OS on your physical machine (your Windows or Mac)

→ → the Hypervisor runs on top of the Host OS in Type 2 setups

- **Guest OS:** the OS running inside the VM (e.g., Ubuntu Linux)

→ → the Guest OS thinks it has real hardware — the Hypervisor tricks it

- **Snapshot:** a point-in-time backup of a VM's entire state

→ → break something → restore snapshot → back to working state in seconds

Hypervisor	Type	Examples	Use Case
Type 1 (Bare Metal)	Production	VMware ESXi, KVM, Xen, Hyper-V (Server)	AWS/Azure data centres — maximum performance
Type 2 (Hosted)	Learning/Dev	VirtualBox, VMware Workstation, Parallels	Developers running Linux VMs on Windows/Mac
Container Engine	Lightweight	Docker, Podman	App isolation without full OS overhead

[BUILDING] REAL-WORLD EXAMPLE

Shopify (Developer Workflow)

Situation:

- Mac developers writing code for Linux production servers — "works on my machine" bugs in production

What they did:

- Standardised all developer machines with Linux VMs matching production environment
- Developers run exact same Ubuntu version locally as production AWS servers
- VMs are snapshotted — new developers get a working environment in minutes, not days

Result:

- Environment-related bugs dropped by 80% — "works on my machine" became "works everywhere"
- Developer onboarding: from 2 days → 30 minutes (restore VM snapshot)

[!] COMMON MISCONCEPTIONS

[X] Myth:

Allocating 100% of RAM to the VM gives it maximum performance

[OK] Reality:

Your Host OS also needs RAM to run the Hypervisor. Rule: give VM 2-4GB max, keep 4GB for Host. Over-allocating causes BOTH to run slowly

[X] Myth:

VMs are the same as Docker containers

[OK] Reality:

VMs virtualise the entire hardware stack (full OS). Containers share the host kernel — much lighter. VMs = full isolation; Containers = process isolation

[>] TRY IT YOURSELF

1. Download VirtualBox (free, [virtualbox.org](https://www.virtualbox.org)) + Ubuntu 22.04 ISO (ubuntu.com/download) — install a local Linux VM
2. Take a VirtualBox snapshot before experimenting — delete a system file — restore snapshot in 10 seconds
3. AWS free tier: launch a t2.micro instance, SSH in, run `uname -a` — you just used Type 1 Hypervisor (KVM) on AWS

[OK] QUICK SUMMARY

- **Virtualization:** divide one physical machine into multiple isolated virtual machines
- **Hypervisor:** software that creates VMs — Type 1 (bare metal, data centres) vs Type 2 (hosted, laptops)
- **VM isolation:** crash one VM — others unaffected — safe for testing dangerous operations
- **Dev-Prod parity:** run same Linux version locally as production → eliminates environment bugs
- **VirtualBox:** free Type 2 Hypervisor — best starting point for running Linux on any laptop
- **Snapshot:** point-in-time backup — experiment freely, restore in seconds if something breaks

[?] MINI QUIZ — Test Your Understanding

Q1. A DevOps engineer needs to test a dangerous script that might crash the OS. What should they use?

- a) Run it directly on their laptop
- b) Run it in a VM — if it crashes the VM, restore a snapshot
- c) Email it to the Ops team to test
- d) Skip testing entirely

[OK] Answer: b — VMs provide safe isolation — a crashed VM is restored in seconds with no damage to the host machine

Q2. AWS uses which type of Hypervisor to create EC2 instances in its data centres?

- a) Type 2 (VirtualBox)
- b) Type 1 (Bare Metal — KVM/Xen)
- c) Type 3 (Container-only)
- d) No Hypervisor — bare metal only

[OK] Answer: b — Data centres use Type 1 Hypervisors (KVM, Xen) installed directly on hardware — maximum performance, no Host OS overhead

Q3. What is a VM Snapshot used for?

- a) Taking a photo of the screen
- b) Saving the entire VM state so you can restore it later
- c) Backing up files to the cloud
- d) Speeding up the VM

[OK] Answer: b — A snapshot saves the entire VM state — disk, memory, settings. Restore it to undo any changes instantly

CHAPTER 03

[B] Beginner

[T] 8 min read

3/9

Exploring the Linux File System

[TARGET] LEARNING OBJECTIVES

By the end of this chapter you will be able to:

- [*] Describe the Linux file system as an upside-down tree starting at / (root)
- [*] Navigate to /etc, /var/log, /home, /bin, /tmp using cd and ls commands
- [*] Explain why "everything is a file" in Linux and give 3 examples
- [*] Fix the top 2 beginner mistakes: backslash vs forward slash, /root vs /

[IDEA] ANALOGY

Linux file system = upside-down tree: / (Root) is the trunk, all directories branch downward — one single starting point, unlike Windows C:\ D:\ E:\ chaos

[*] DID YOU KNOW? [*]

/

Single root — entire OS starts here

~

Tilde = shortcut to your home directory

.

Hidden files start with a dot (.bashrc)

Linux Directory Hierarchy — Everything starts at /



e.g., /etc/nginx/nginx.conf /var/log/syslog /home/aryashree/.bashrc

Linux Directory Hierarchy — / is the root, all paths branch from here. Everything including devices is a file

Key Directories — What Lives Where

- **/ (Root):** Starting point of the entire file system — the trunk of the tree
- **/bin:** Binary programs — core commands: ls, cd, mkdir, cp, mv, rm
- **/etc:** Configuration files — nginx.conf, ssh/sshd_config, cron jobs
- **/home:** User personal files — /home/aryashree is your home folder
- **/var:** Variable data — logs (/var/log/), databases, spools, mail

- **/tmp:** Temporary files — deleted on reboot — safe scratch space
- **/usr:** User-installed software, shared libraries, documentation
- **/proc:** Virtual filesystem — running processes and kernel info (not real files)
- **/dev:** Device files — /dev/sda is your hard drive, /dev/null is the trash

Key Concepts

- **Everything is a file:** Linux treats everything as a file
 - → /dev/sda = your hard drive (a file)
 - → /dev/null = trash bin (a file you can write to)
 - → /proc/cpuinfo = your CPU information (a file)
- **Hidden files:** files starting with . are hidden from normal view
 - → .bashrc = bash config, .ssh/authorized_keys = SSH keys
 - → show with: ls -a (the -a flag means "all including hidden")
- **Absolute vs Relative paths:**
 - → Absolute: starts from / — e.g., /etc/nginx/nginx.conf (always works)
 - → Relative: starts from current position — e.g., ../config (depends on where you are)
 - → ~ expands to your home directory — cd ~ = go home

Command	What It Does	Example	Tip
pwd	Print Working Directory — where am I?	pwd → /var/log	First command to run when lost
ls -la	List All files including hidden + permissions	ls -la /etc	l=long, a=all
cd /path	Change Directory to absolute path	cd /var/log/nginx	Use Tab to autocomplete
cd ~	Go to your home directory	cd ~	Shortcut from anywhere
cd ..	Go up one directory level	cd ../../	Two levels up
find / -name	Search entire system for file	find / -name nginx.conf	Slow but thorough

[BUILDING] REAL-WORLD EXAMPLE**Real DevOps Incident****Situation:**

- Web server stopped serving pages — error 502. Engineer needs to diagnose fast

What they did:

- `cd /var/log/nginx` to navigate to Nginx log directory
- `cat error.log` to read the error — found "upstream connection refused"
- `cd /etc/nginx` to check config — found typo in `nginx.conf` upstream block
- Fixed typo → `sudo nginx -t` (test config) → `sudo systemctl reload nginx`

Result:

- Site back online in 4 minutes — entire diagnosis done via file system navigation

[!] COMMON MISCONCEPTIONS**[X] Myth:**

Using backslashes (`\`) like Windows paths in Linux

[OK] Reality:

Linux uses FORWARD slashes (`/`). Always: `/etc/nginx` — never: `\\etc\\nginx`. Backslash in Linux means "escape the next character" — completely different meaning

[X] Myth:

Confusing `/root` with `/` (root directory)

[OK] Reality:

`/` is the base of the entire file system. `/root` is just the home folder for the root (admin) user. They are completely different locations

[>] TRY IT YOURSELF

1. In your Linux terminal: type `pwd` → then `cd /` → then `ls -la` to see all root-level directories
2. Navigate to `/var/log` and run `ls -la` — identify at least 3 log files. Run `cat syslog | head -20` to read the system log
3. Find all `.conf` files in `/etc`: run `find /etc -name "*.conf" 2>/dev/null | head -20`

[OK] QUICK SUMMARY

- **/ (Root):** single starting point of all Linux paths — the trunk of the upside-down tree
- **Key dirs:** `/etc` = configs, `/var/log` = logs, `/home` = users, `/bin` = commands, `/tmp` = temp
- **Everything is a file:** devices, processes, network — all represented as files in `/dev` or `/proc`
- **Hidden files:** start with `.` (dot) — visible with `ls -a` — e.g., `.bashrc`, `.ssh/authorized_keys`
- **Forward slashes only:** `/etc/nginx` (correct) — never backslash like Windows
- **~ shortcut:** always expands to your home directory — `cd ~` = instant home navigation

[?] MINI QUIZ — Test Your Understanding**Q1. Where would you find Nginx web server configuration files on Linux?**

- a) /home/nginx
- b) /var/log/nginx
- c) /etc/nginx
- d) /tmp/nginx

[OK] Answer: c — /etc contains all system configuration files. /etc/nginx/nginx.conf is the standard location for Nginx config

Q2. Your disk shows usage at 100%. Which directory should you check first for oversized log files?

- a) /etc
- b) /var/log
- c) /home
- d) /bin

[OK] Answer: b — /var/log stores all system and application logs — they grow continuously and are the most common cause of disk full issues

Q3. What does the ~ (tilde) represent in a Linux file path?

- a) Root directory /
- b) Parent directory ..
- c) Current user home directory
- d) Temporary directory /tmp

[OK] Answer: c — ~ always expands to the current logged-in user home directory — e.g., /home/aryashree. cd ~ takes you home from anywhere

CHAPTER 04

[B] Beginner

[T] 7 min read

4/9

Managing Software on Linux

[TARGET] LEARNING OBJECTIVES

By the end of this chapter you will be able to:

- [*] Explain the Package Manager concept using the "App Store" analogy
- [*] Use apt (Ubuntu) and yum/dnf (CentOS) to install, update, and remove software
- [*] Explain what a Repository is and why you should always run apt update first
- [*] Install a web server (nginx) using a single command

[IDEA] ANALOGY

Package Manager = Linux App Store: instead of googling .exe files, you type a command
→ Linux downloads software from a trusted official library (Repository) with all dependencies automatically

[*] DID YOU KNOW? [*]

1 cmd

`sudo apt install nginx` — installs + starts web server

Auto

dependencies installed automatically — no manual steps

Trusted

packages signed & from official sources only

Package Managers — apt vs yum

- **apt (Advanced Package Tool):** Ubuntu and Debian-based distros
 - → `sudo apt update` — refresh list of available packages (sync with repo)
 - → `sudo apt install nginx` — download + install + configure in one command
 - → `sudo apt remove nginx` — uninstall the software
 - → `sudo apt upgrade` — update ALL installed packages to latest versions
- **yum / dnf:** CentOS, RHEL, Fedora-based distros
 - → `sudo yum install nginx` (older CentOS) or `sudo dnf install nginx` (newer)
 - → dnf is the modern replacement for yum — same syntax, better performance
- **Repository (Repo):** official online library where packages are stored
 - → Ubuntu connects to Ubuntu servers: `archive.ubuntu.com`
 - → `/etc/apt/sources.list` — file that lists all your active repositories
 - → you can add custom repos: e.g., Docker repo, Kubernetes repo
- **Dependencies:** other software a package needs to work
 - → install Chrome → Package Manager installs `libssl`, `libglib`, `libX11` automatically
 - → this is why apt is far better than downloading .exe files manually

Command	Distro	What It Does
<code>sudo apt update</code>	Ubuntu/Debian	Refresh package list from repos (ALWAYS run first)

Command	Distro	What It Does
<code>sudo apt install git</code>	Ubuntu/Debian	Install Git version control tool
<code>sudo apt remove git</code>	Ubuntu/Debian	Uninstall Git
<code>sudo apt upgrade</code>	Ubuntu/Debian	Update all installed packages to latest
<code>sudo dnf install git</code>	CentOS/RHEL/Fedora	Install Git on Red Hat-based systems
<code>sudo apt install apache2 -y</code>	Ubuntu/Debian	-y flag auto-answers Yes to all prompts

[BUILDING] REAL-WORLD EXAMPLE**AWS DevOps Setup****Situation:**

- Fresh Ubuntu 22.04 EC2 server — needs Nginx web server, Docker, and Git installed within 5 minutes

What they did:

- `sudo apt update` — sync package list (always first)
- `sudo apt install nginx git docker.io -y` — install 3 tools in one command
- `systemctl status nginx` — verify Nginx is running

Result:

- Complete web server setup in under 2 minutes — 3 separate tools, one command
- Compare: Windows Server install would take 30+ minutes with multiple installer wizards

[!] COMMON MISCONCEPTIONS**[X] Myth:**

Forgetting to run `apt update` before installing

[OK] Reality:

Without update, apt uses a stale local cache. You get "Package not found" errors or install outdated versions. ALWAYS run `sudo apt update` first — takes 10 seconds

[X] Myth:

`sudo apt upgrade` will break my server

[OK] Reality:

In production, upgrade carefully — test in staging first. But on a fresh server or dev VM, upgrade is safe and

[>] TRY IT YOURSELF

1. Run: `sudo apt update && sudo apt install nginx -y` → then open your browser to `http://localhost` — you should see the Nginx welcome page
2. Check what version was installed: `nginx -v`. Then stop it: `sudo systemctl stop nginx`. Then start it again: `sudo systemctl start nginx`
3. Install the tree command: `sudo apt install tree -y` → run `tree /etc/nginx` to see the config directory structure visually

[OK] QUICK SUMMARY

- **Package Manager:** Linux App Store — downloads software from trusted repos with all dependencies
- **apt:** Ubuntu/Debian package manager — `sudo apt update` then `sudo apt install [package]`
- **yum/dnf:** CentOS/RHEL package manager — same concept, different command
- **Repository:** official online library of packages — `/etc/apt/sources.list` lists yours
- **Always update first:** `sudo apt update` before installing — gets latest package list
- **Dependencies:** auto-installed — no manual "you also need libX11" errors

[?] MINI QUIZ — Test Your Understanding

Q1. What does `sudo apt update` actually do?

- a) Updates all installed software to latest versions
- b) Refreshes the local list of available packages from repositories
- c) Installs security patches
- d) Reboots the server

[OK] Answer: b — `apt update` syncs the local package index with remote repos — it does NOT install anything. `sudo apt upgrade` installs the updates

Q2. A DevOps engineer needs `git`, `docker`, and `nginx` on a new Ubuntu server. Most efficient method?

- a) Three separate commands, one per tool
- b) `sudo apt update && sudo apt install git docker.io nginx -y` — all in one line
- c) Download each from official websites
- d) Use the Ubuntu App Store GUI

[OK] Answer: b — `apt install` accepts multiple package names in one command — efficient, automated, consistent

Q3. Which file lists all active package repositories on Ubuntu?

- a) `/etc/packages.list`
- b) `/var/apt/sources`
- c) `/etc/apt/sources.list`
- d) `/home/apt/repos`

[OK] Answer: c — `/etc/apt/sources.list` contains all active repository URLs — this is where `apt` looks for packages

CHAPTER 05

[B] Beginner

[T] 6 min read

5/9

Working with Text Editors (Nano & Vim)

[TARGET] LEARNING OBJECTIVES

By the end of this chapter you will be able to:

- [*] Explain why CLI text editors are essential for server management
- [*] Open, edit, save, and exit files in both Nano and Vim
- [*] Switch between Command Mode and Insert Mode in Vim
- [*] Fix a typo in a production config file using vi /etc/nginx/nginx.conf

[IDEA] ANALOGY

Nano = Notepad (simple, instructions at bottom). Vim = pilot cockpit (confusing at first, but once learned — fastest editor on any Linux system on Earth)

[*] DID YOU KNOW? [*]

Vim

pre-installed on every Linux system by default

i

one key to switch to Insert mode in Vim

:wq

three chars to save and quit Vim

Nano — The Beginner-Friendly Editor

- **Open a file:** nano filename.txt — creates if not exists
- **Write text:** just start typing — Nano is WYSIWYG (What You See Is What You Get)
- **Save:** Ctrl+O then Enter — the ^ in the menu means Ctrl
- **Exit:** Ctrl+X — prompts to save if unsaved changes exist
- **Search:** Ctrl+W then type search term
- **Cut line:** Ctrl+K — paste with Ctrl+U

Vim — The Universal Linux Editor

- **Vim has two modes — this is the source of all confusion:**
 - → Command Mode (default when you open Vim): navigate, delete, copy, search
 - → Insert Mode (press i to enter): type and edit text like a normal editor
 - → Press Esc to return to Command Mode from Insert Mode
- **Essential Vim workflow — 5 commands to know first:**
 - → vim filename.txt — open file
 - → i — enter Insert Mode (now you can type)
 - → Esc — return to Command Mode
 - → :w — write (save) the file

- → :q — quit (only works if no unsaved changes)
- → :wq — write and quit (save and exit) — most common
- → :q! — force quit WITHOUT saving (emergency exit)

- **Why learn Vim despite the learning curve:**

- → pre-installed on every Linux — even minimal Alpine containers
- → faster navigation once fluent: gg (top), G (bottom), dd (delete line)
- → SSH into a broken server with no GUI — Vim is always there

Action	Nano Command	Vim Command
Open file	nano config.txt	vim config.txt or vi config.txt
Enter edit mode	Just start typing	Press i (Insert Mode)
Save file	Ctrl+O, Enter	Esc then :w
Save and quit	Ctrl+X	Esc then :wq
Quit without saving	Ctrl+X then N	Esc then :q!
Search text	Ctrl+W	/ then search term, n for next
Delete line	Ctrl+K	dd (in Command Mode)

[BUILDING] REAL-WORLD EXAMPLE

Real Production Incident

Situation:

- Nginx returning 502 on an AWS server — typo in upstream config block found via logs

What they did:

- SSH into server: `ssh ubuntu@aws-server-ip`
- Open config: `vi /etc/nginx/nginx.conf`
- Press i to enter Insert Mode — fix the typo
- Press Esc → :wq to save and quit → `sudo nginx -t` to test config
- `sudo systemctl reload nginx` — apply changes with zero downtime

Result:

- Server recovered in 3 minutes — entire fix done with Vim over SSH
- Without Vim knowledge: would have needed GUI access or spent 20 minutes with Nano

[!] COMMON MISCONCEPTIONS**[X] Myth:**

"I got stuck in Vim and cannot exit" — the universal beginner experience

[OK] Reality:

The escape: Press Esc (multiple times if needed) → then type :q! → then Enter. This force-quits without saving. Pin this to your monitor

[X] Myth:

"Nano is better because Vim is too complex"

[OK] Reality:

Nano is better for beginners. Vim is better for professionals. The goal is to eventually learn Vim — it is on every Linux system, faster for complex edits, and standard in DevOps workflows

[>] TRY IT YOURSELF

1. Create a file with Nano: `nano my_notes.txt` → type 3 lines → save with `Ctrl+O` → exit with `Ctrl+X` → verify with `cat my_notes.txt`
2. Open the same file in Vim: `vim my_notes.txt` → press `i` → add a line → press `Esc` → type `:wq` → Enter to save
3. Simulate a production edit: `sudo vim /etc/hosts` → press `i` → add "127.0.0.1 myapp.local" → `Esc` → `:wq` → verify with `cat /etc/hosts`

[OK] QUICK SUMMARY

- **Nano:** beginner-friendly — WYSIWYG, instructions at bottom, `Ctrl+O` save, `Ctrl+X` exit
- **Vim:** two modes — Command Mode (default, navigate/delete) and Insert Mode (type text)
- **Switch modes:** press `i` to enter Insert, press `Esc` to return to Command Mode
- **Vim save/quit:** `Esc` → `:wq` (save+quit), `:q!` (force quit no save)
- **Why Vim matters:** pre-installed everywhere — the only editor guaranteed on any Linux server
- **Real use:** SSH into production server → `vi /etc/nginx/nginx.conf` → fix typo → `:wq` → reload

[?] MINI QUIZ — Test Your Understanding

Q1. You opened a file in Vim and want to type text. What do you press first?

- a) Ctrl+I
- b) T for Type
- c) i (to enter Insert Mode)
- d) Start typing immediately

[OK] Answer: c — Vim opens in Command Mode by default. Press i to enter Insert Mode — only then can you type text normally

Q2. You made an error in Vim and want to exit WITHOUT saving changes. Which command?

- a) :wq
- b) :exit
- c) Ctrl+Z
- d) :q! (force quit without saving)

[OK] Answer: d — :q! forces Vim to quit ignoring unsaved changes — the emergency exit that every beginner must memorise

Q3. Which editor is always available on EVERY Linux system including minimal Docker containers?

- a) Nano
- b) Gedit
- c) VSCode
- d) Vi/Vim

[OK] Answer: d — Vi/Vim is installed by default on virtually every Linux system. Nano must be installed separately — it is not always available

CHAPTER 06

[I] Intermediate

[T] 9 min read

6/9

Linux User and Permission Management

[TARGET] LEARNING OBJECTIVES

By the end of this chapter you will be able to:

- [*] Decode any Linux permission string (e.g., -rwxr-xr--) into owner/group/others
- [*] Use chmod to set file permissions using both symbolic and numeric (755, 644) methods
- [*] Explain the difference between sudo and root and when to use each
- [*] Create a new user and add them to the sudo group

[IDEA] ANALOGY

Linux = secure office building: Users = staff with ID badges, Groups = departments (HR, IT, Finance), Permissions = door access rules, Root = building owner with master key to everything

[*] DID YOU KNOW? [*]

rwx

Read(4) Write(2) Execute(1) — 3 permission types

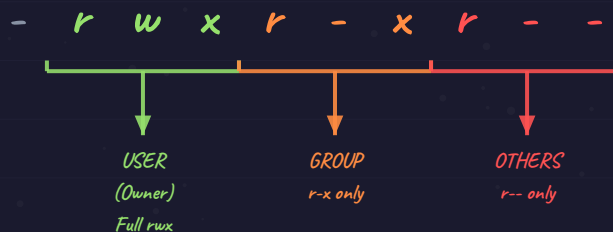
ugo

User, Group, Others — 3 owner levels

755

Most common permission for executables & dirs

Linux Permission String — -rwxr-xr--



Write (2)



Read(4)

Permission string breakdown: first char=file type, then 3 chars each for User/Group/Others — r=read w=write x=execute

Understanding Permission Strings

- **Permission string format: -rwxr-xr-- (10 characters)**
 - → Char 1: file type — (-) regular file, (d) directory, (l) symlink
 - → Chars 2-4: User (owner) permissions — rwx = full read/write/execute
 - → Chars 5-7: Group permissions — r-x = read and execute, no write
 - → Chars 8-10: Others permissions — r-- = read only
- **Numeric (Octal) permissions: r=4, w=2, x=1**

- → 755 = User:rw(7) Group:r-x(5) Others:r-x(5) — standard for executables
- → 644 = User:rw(6) Group:r--(4) Others:r--(4) — standard for config files
- → 777 = everyone can do everything — DANGEROUS, avoid in production
- → calculate: $rw = 4 + 2 + 1 = 7$, $r-x = 4 + 0 + 1 = 5$, $r-- = 4 + 0 + 0 = 4$

- **sudo (SuperUser DO):** temporary root privileges for one command

- → sudo apt install nginx — run as root without being root
- → logged in audit trail: /var/log/auth.log records every sudo command
- → safer than logging in as root — limits blast radius of mistakes

Command	What It Does	Example
chmod 755 file.sh	Set rwxr-xr-x — standard executable	chmod 755 deploy.sh
chmod +x file.sh	Add execute permission to all	chmod +x deploy.sh
chmod 644 config	Set rw-r--r-- — standard config file	chmod 644 nginx.conf
chown user:group file	Change file owner and group	chown www-data:www-data /var/www
sudo useradd devops	Create new user named devops	sudo useradd -m devops
sudo usermod -aG sudo devops	Add devops user to sudo group	sudo usermod -aG sudo devops

[BUILDING] REAL-WORLD EXAMPLE

Real Security Incident Prevention

Situation:

- Script deploy.sh created — runs fine locally but returns "Permission denied" on server

What they did:

- ls -la deploy.sh — showed -rw-r--r-- (no execute permission)
- chmod +x deploy.sh — added execute permission
- ls -la again — confirmed -rwxr-xr-- (execute added for owner)
- ./deploy.sh — script runs successfully

Result:

- Deployment unblocked — 30-second fix vs 30-minute debugging session

[!] COMMON MISCONCEPTIONS

[X] Myth:

Running everything as root (sudo su -) for convenience

[OK] Reality:

Root has no restrictions. One typo as root (rm -rf /) destroys the entire server. Best practice: use a normal user, sudo only for specific commands that require it — full audit trail maintained

[X] Myth:

chmod 777 fixes any permission problem

[OK] Reality:

777 gives EVERYONE on the system full read/write/execute access — critical security vulnerability. Fix permissions properly with 755 (executables) or 644 (config files)

[>] TRY IT YOURSELF

1. Create a script: `echo "echo hello" > test.sh` → try running it (`./test.sh` — fails) → `chmod +x test.sh` → run again (succeeds) — understand why
2. Decode this string: `drwxr-x---` Answer: `d=directory, owner=rwx, group=r-x, others=no access (---)`
3. Create a new user: `sudo useradd -m testuser` → `sudo passwd testuser` → `su testuser` → `whoami` — verify you switched users

[OK] QUICK SUMMARY

- **Permission string:** `-rwxr-xr--` = file type + User(3) + Group(3) + Others(3)
- **r=4, w=2, x=1:** `755=rwxr-xr-x` (executables), `644=rw-r--r--` (config files)
- **chmod:** change permissions — `chmod +x` adds execute, `chmod 755` sets numeric
- **chown:** change file owner — `chown user:group filename`
- **sudo:** temporary root for one command — safer than logging in as root
- **Never chmod 777:** gives everyone full access — serious security vulnerability

[?] MINI QUIZ — Test Your Understanding

Q1. What permissions does `chmod 755` set on a file?

- a) Everyone full access (`rwxrwxrwx`)
- b) Owner: `rwx`, Group: `r-x`, Others: `r-x`
- c) Read-only for everyone
- d) Owner: `rw-`, Group: `r--`, Others: `r--`

[OK] Answer: b — `755 = 7(rwx)+5(r-x)+5(r-x)`. Standard for scripts and executables — owner can run/edit, others can only run

Q2. A new script fails with "Permission Denied". What is the fastest fix?

- a) `chmod 777 the-script.sh`
- b) `sudo the-script.sh`
- c) `chmod +x the-script.sh`
- d) Run as root

[OK] Answer: c — `chmod +x` adds execute permission for all users — the minimal, correct fix for a script permission error

Q3. What does `sudo` do?

- a) Creates a new superuser account
- b) Permanently grants root access
- c) Runs one command with root privileges, then returns to normal user
- d) Disables all permission checks

[OK] Answer: c — `sudo = SuperUser DO` — runs only the specified command as root, then returns to normal permissions. Every `sudo` call is logged

CHAPTER 07

[I] Intermediate

[T] 8 min read

7/9

Mastering the Command Line

[TARGET] LEARNING OBJECTIVES

By the end of this chapter you will be able to:

- [*] Use pipes (|) to chain commands and create powerful one-liners
- [*] Redirect output to files with > (overwrite) and >> (append)
- [*] Search log files with grep to find specific errors in seconds
- [*] Build complex pipelines: `cat | grep | sort | > file.txt`

[IDEA] ANALOGY

Command line = LEGO blocks — each command does one thing well. Pipes (|) connect them into powerful workflows. *grep "Error" server.log* finds needles in haystacks instantly

[*] DID YOU KNOW? [*]

|

Pipe: pass output of A as input to B

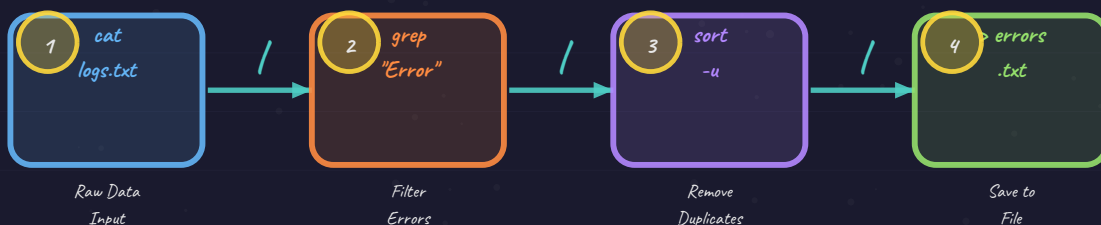
>

Redirect: save command output to a file

grep

Search text — finds any pattern in any file

The Assembly Line — Linux Command Pipeline



The Assembly Line: `cat | grep | sort | >` — data flows left to right, each command processes and passes to the next

The Power Trio: Pipes, Redirection, Grep

- **Pipe (|):** pass the output of one command as the input to the next
 - → `cat access.log | grep "404"` — find all 404 errors in the access log
 - → `ls -la | grep ".conf"` — list only .conf files
 - → `ps aux | grep nginx` — find nginx processes
 - → you can chain unlimited pipes: `cmd1 | cmd2 | cmd3 | cmd4`
- **Redirection (> and >>):** save output to files instead of screen
 - → > (overwrite): `command > file.txt` — replaces file contents completely
 - → >> (append): `command >> file.txt` — adds to end of file without deleting

- → 2> error.txt — redirect error messages (stderr) to file
- → command > /dev/null 2>&1 — discard ALL output (silence command)

- **grep:** search text files for patterns

- → grep "error" server.log — case-sensitive search
- → grep -i "error" server.log — case-insensitive (-i flag)
- → grep -r "database" /etc/ — recursive search through all files
- → grep -n "error" file.log — show line numbers with results

- **Essential commands for DevOps log analysis:**

- → tail -f /var/log/nginx/access.log — LIVE log stream (Ctrl+C to stop)
- → du -h --max-depth=1 | sort -h — find which directories use most disk
- → history | grep docker — find all docker commands you've run
- → wc -l file.txt — count number of lines in a file

Command	Purpose	Example	Output
grep "error" file	Search file for pattern	grep "ERROR" app.log	Lines containing ERROR
cat file grep X	Read file, filter lines	cat access.log grep 404	All 404 lines
command > file	Overwrite file with output	ls > filelist.txt	Creates/replaces filelist.txt
command >> file	Append output to file	echo "done" >> deploy.log	Adds to end of log
tail -f logfile	Live stream of log file	tail -f /var/log/syslog	Real-time log viewing
du -h sort -h	Disk usage sorted by size	du -h sort -h	Folders from smallest to largest

[BUILDING] REAL-WORLD EXAMPLE

Real Debugging Session

Situation:

- Customer reports 404 errors on website — engineer needs to find all 404s and what URLs they hit

What they did:

- grep "404" /var/log/nginx/access.log | wc -l — counted total 404s
- grep "404" /var/log/nginx/access.log | awk '{print \$7}' | sort | uniq -c | sort -rn | head -20
- Found /api/v1/users/profile returning 404 — API route not configured

Result:

- Found root cause in 45 seconds using 3 piped commands
- Same analysis manually would take 30+ minutes scrolling through 100,000 log lines

[!] COMMON MISCONCEPTIONS**[X] Myth:**

Confusing `>` (overwrite) with `>>` (append)

[OK] Reality:

`>` completely REPLACES the file content. `>>` ADDS to the end. Accidentally using `>` on a log file destroys all existing logs. When in doubt: use `>>` for logs, `>` only when you intentionally want to reset

[X] Myth:

"Pipes are advanced — I will learn them later"

[OK] Reality:

Pipes are the foundation of Linux power. Every DevOps engineer uses `cat | grep | sort | uniq` daily. Learn them in week 1, not later. One pipeline replaces hours of manual searching

[>] TRY IT YOURSELF

1. Create a test file: `for i in {1..20}; do echo "Line $i: $([ $((${i%3}) -eq 0 ] && echo ERROR || echo OK)"; done > test.log` → then `grep "ERROR" test.log`
2. Count errors: `grep "ERROR" test.log | wc -l` → then save only errors: `grep "ERROR" test.log > errors_only.txt`
3. Disk analysis: `du -h /var/log 2>/dev/null | sort -h | tail -10` — see which log files are biggest

[OK] QUICK SUMMARY

- **Pipe (`|`):** connects commands — output of left becomes input of right: `cat log | grep error`
- **`>` (overwrite):** saves command output to file, replacing all existing content
- **`>>` (append):** adds command output to end of file — does not delete existing data
- **`grep`:** search text for patterns — `grep -i` (case insensitive) `-r` (recursive) `-n` (line numbers)
- **`tail -f`:** live log stream — essential for monitoring running applications
- **Pipeline power:** `cat log | grep ERROR | sort | uniq -c | sort -rn` — complex in one line

[?] MINI QUIZ — Test Your Understanding**Q1. What does the | (pipe) do in: cat server.log | grep "ERROR"?**

- a) Runs both commands simultaneously
- b) Passes output of cat as input to grep
- c) Creates a pipe file on disk
- d) Separates two unrelated commands

[OK] Answer: b — Pipe passes stdout of left command to stdin of right — cat reads the file, grep filters it. Chaining creates powerful single-line workflows

Q2. You want to add entries to a log file without deleting existing content. Which operator?

- a) > (overwrite)
- b) | (pipe)
- c) >> (append)
- d) < (input redirect)

[OK] Answer: c — >> appends to end of file without affecting existing content. > destroys all existing content — dangerous for logs

Q3. Your disk is full. Which command identifies which directories use the most space?

- a) ls -la / | head -20
- b) grep -r "big" /
- c) du -h --max-depth=1 | sort -h
- d) find / -size +1G

[OK] Answer: c — du -h shows human-readable sizes, --max-depth=1 shows only top-level dirs, sort -h sorts by size. Standard first-response to disk full incidents

CHAPTER 08

[I] Intermediate

[T] 9 min read

8/9

Introduction to Shell Scripting

[TARGET] LEARNING OBJECTIVES

By the end of this chapter you will be able to:

- [*] Write a shell script with shebang, variables, and basic logic
- [*] Make a script executable and run it with `./script.sh`
- [*] Schedule automated tasks with cron (e.g., backup every night at 3 AM)
- [*] Build a real backup script that creates a compressed archive

[IDEA] ANALOGY

Shell script = Robot To-Do List: write the instructions once, give the list to the robot (Linux), it performs all steps forever automatically — never forgets, never makes typos

[*] DID YOU KNOW? [*]

Automate

repeat same 10 commands every day — script it once

Cron

schedule scripts to run at exact times automatically

#!/bin/bash

shebang line — tells Linux which interpreter to use

Writing Your First Shell Script

- **Shebang (#!/bin/bash):** first line — tells Linux which interpreter runs the script
 - → `#!/bin/bash` — use bash (most common)
 - → `#!/bin/sh` — use POSIX sh (more portable)
 - → `#!/usr/bin/env python3` — use Python (same concept for Python scripts)
 - → without shebang: Linux guesses the interpreter — often fails
- **Variables:** store and reuse values
 - → `NAME="Aryashree"` — assign (no spaces around =)
 - → `echo $NAME` — use the variable (\$NAME expands to value)
 - → `DATE=$(date +%Y-%m-%d)` — capture command output into variable
 - → `$1, $2` — script arguments: `./deploy.sh production` → `$1 = "production"`
- **Script execution steps:**
 - → Step 1: create — `vim backup.sh`
 - → Step 2: make executable — `chmod +x backup.sh`
 - → Step 3: run — `./backup.sh` (or `bash backup.sh`)
- **Cron:** task scheduler — runs scripts automatically on a schedule
 - → `crontab -e` — open your personal cron schedule
 - → format: [min] [hour] [day] [month] [weekday] [command]
 - → `0 3 * * * /home/aryashree/backup.sh` — every day at 3:00 AM
 - → `* /5 * * * * /scripts/check_disk.sh` — every 5 minutes

→ → crontab -l — list all your scheduled cron jobs

Real Backup Script (Study This)

backup.sh — backs up /data, compresses, removes old backups:

```
#!/bin/bash
# Backup script — runs nightly via cron
DATE=$(date +%Y-%m-%d)
BACKUP_DIR="/backups"
SOURCE_DIR="/data"
BACKUP_FILE="$BACKUP_DIR/backup-$DATE.tar.gz"
# Create backup directory if it does not exist
mkdir -p $BACKUP_DIR
# Create compressed archive
tar -czf $BACKUP_FILE $SOURCE_DIR
# Delete backups older than 7 days
find $BACKUP_DIR -name "*.tar.gz" -mtime +7 -delete
echo "Backup complete: $BACKUP_FILE"
```

[BUILDING] REAL-WORLD EXAMPLE

Cron Job — Real Production Use

Situation:

- Hard drive fills up every month with Nginx logs — engineer manually cleans it each time

What they did:

- Wrote a 5-line shell script: `find /var/log/nginx -name "*.log" -mtime +30 -delete`
- Added to crontab: `0 2 * * 1 /scripts/cleanup_logs.sh` (every Monday at 2 AM)
- Added email alert: `mail -s "Log cleanup complete" admin@company.com` at end of script

Result:

- Disk usage problem solved permanently — runs automatically every week

[!] COMMON MISCONCEPTIONS

[X] Myth:

"Permission Denied" when running a script

[OK] Reality:

You must run `chmod +x script.sh` before `./script.sh`. The script file exists but has no execute permission by default. Always `chmod +x` after creating a new script

[X] Myth:

"Scripts are for advanced engineers — I am a beginner"

[OK] Reality:

If you do ANY task more than twice manually, it deserves a script. A 5-line bash script that automates one daily task is a legitimate DevOps contribution. Start simple, grow complexity gradually

[>] TRY IT YOURSELF

1. Write `hello_world.sh`: `#!/bin/bash` then `echo "Hello $USER, today is $(date)"`. Make executable with `chmod +x` → run with `./hello_world.sh`
2. Write the `backup.sh` script above → test it with `./backup.sh` → check `/backups/` for the created archive
3. Schedule it in cron: `crontab -e` → add `0 3 * * * /full/path/to/backup.sh` → verify with `crontab -l`

[OK] QUICK SUMMARY

- **#!/bin/bash**: shebang — first line of every script — declares the interpreter
- **Variables**: `NAME="value"` to assign, `$NAME` to use — no spaces around =
- **chmod +x**: must add execute permission before running any script
- **\$(command)**: captures command output into variable — `DATE=$(date +%Y-%m-%d)`
- **Cron**: task scheduler — format: `[min][hour][day][month][weekday][command]`
- **Automate rule**: anything done more than twice manually deserves a bash script

[?] MINI QUIZ — Test Your Understanding

Q1. What is the purpose of the `#!/bin/bash` line at the top of a script?

- a) A comment that describes the script
- b) The shebang — tells Linux to use bash to interpret the script
- c) The script name
- d) Required import statement

[OK] Answer: b — The shebang (#!) tells the OS which interpreter to use. Without it, Linux guesses and often fails. Always include it as line 1

Q2. A cron job is set to: `0 3 * * * /scripts/backup.sh`. When does it run?

- a) Every 3 minutes
- b) 3 times per day
- c) Every day at 3:00 AM
- d) Every 3 hours

[OK] Answer: c — Cron format: [0=min] [3=hour] [*=any day] [*=any month] [*=any weekday] = 3:00 AM every single day

Q3. Your script fails with "Permission Denied". The correct fix is:

- a) Run it with `sudo ./script.sh`
- b) Move it to `/bin/`
- c) `chmod +x script.sh` then run `./script.sh`
- d) Rename it to `.bash` extension

[OK] Answer: c — Scripts need execute permission (chmod +x) before they can run. This is intentional — prevents accidental execution of text files

CHAPTER 09

[I] Intermediate

[T] 10 min read

9/9

Environment Variables, Networking & SSH

[TARGET] LEARNING OBJECTIVES

By the end of this chapter you will be able to:

- [*] Create and export environment variables and explain their use in DevOps secrets management
- [*] Explain IP, Port, and DNS using the "office building" analogy
- [*] SSH into a remote server using both password and key-based authentication
- [*] Generate SSH keys and add your public key to an authorised_keys file

[IDEA] ANALOGY

Environment variables = ID badge: logged-in user carries `USER=john HOME=/home/john EDITOR=vim` — every program reads the badge to know your settings. SSH = magic encrypted tunnel to any server worldwide

[*] DID YOU KNOW? [*]

\$VAR

dollar sign accesses variable value

Port 22

default SSH port — all Linux remote access

NEVER

share your SSH private key (.pem)

Environment Variables

- **What they are:** named key=value pairs that programs read for configuration
 - → `DB_PASSWORD=secret123` — app reads this instead of hardcoded passwords
 - → `ENV=production` — switches app behaviour based on environment
 - → `JAVA_HOME=/usr/lib/jvm/java-17` — tells Java apps where Java is installed
 - → `PATH=/usr/bin:/usr/local/bin:...` — list of directories Linux searches for programs
- **Creating and using variables:**
 - → `MY_VAR="Hello"` — creates variable (only in current session)
 - → `export MY_VAR="Hello"` — makes variable visible to child processes
 - → `echo $MY_VAR` — prints the value
 - → `env` or `printenv` — list ALL current environment variables
 - → `unset MY_VAR` — delete the variable
- **Making variables permanent:**
 - → add to `~/.bashrc` — loads every time you open a terminal
 - → add to `/etc/environment` — system-wide, all users (use for PATH)
 - → variables in current session only last until terminal closes

Networking Essentials

- **IP Address:** unique network identifier for every computer
 - → analogy: the building street address — gets you to the right machine

- → 192.168.x.x — private/local network (inside your home/office)
- → 203.0.113.x — public IP (internet-visible, assigned by your ISP)
- → 127.0.0.1 — localhost = "this computer" — always safe

- **Port:** specific service running on an IP address
 - → analogy: office number inside the building
 - → Port 80: HTTP web traffic, Port 443: HTTPS (encrypted web)
 - → Port 22: SSH (remote Linux access), Port 3306: MySQL database
 - → IP:Port together: 192.168.1.10:80 = web server on that machine
- **DNS (Domain Name System):** translates domain names to IP addresses
 - → analogy: address book — google.com → 142.250.80.46
 - → without DNS you would type IP addresses into your browser
 - → /etc/hosts — local override file (used for local dev: 127.0.0.1 myapp.local)
 - → dig google.com or nslookup google.com — see DNS resolution

SSH — Secure Shell (The DevOps Superpower)

- **What SSH does:** encrypted tunnel to control any Linux server remotely
 - → you type commands on your laptop → executed on server in Tokyo → results return
 - → encrypted end-to-end: even your ISP cannot see what you type
 - → 99% of DevOps remote work is done via SSH — zero physical server access
- **Password-based SSH (less secure but easy to start):**
 - → ssh username@ip-address — then enter password
 - → example: ssh ubuntu@54.123.45.67 (connect to AWS server)
 - → problem: passwords can be brute-forced, forgotten, or stolen
- **Key-based SSH (the professional standard):**
 - → ssh-keygen -t rsa -b 4096 — generate a key pair
 - → creates ~/.ssh/id_rsa (PRIVATE — never share) and ~/.ssh/id_rsa.pub (PUBLIC — share freely)
 - → copy public key to server: ssh-copy-id ubuntu@server-ip
 - → or manually: cat ~/.ssh/id_rsa.pub → paste into server's ~/.ssh/authorized_keys
 - → AWS EC2: download the .pem key → ssh -i key.pem ubuntu@ec2-ip



⚡ SSH tunnel: local command → encrypted through tunnel → executed on remote server → encrypted result returns

Concept	Command	Example	Purpose
Create variable	<code>export VAR="value"</code>	<code>export DB_PASS="secret"</code>	Available to all child processes
Read variable	<code>echo \$VAR</code>	<code>echo \$DB_PASS</code>	Print variable value
List all vars	<code>env</code> or <code>printenv</code>	<code>env grep DB</code>	See all environment variables
SSH password	<code>ssh user@ip</code>	<code>ssh ubuntu@54.1.2.3</code>	Basic remote access
SSH key-based	<code>ssh -i key.pem user@ip</code>	<code>ssh -i aws.pem ubuntu@server</code>	More secure, AWS standard
Generate keys	<code>ssh-keygen -t rsa</code>	<code>ssh-keygen -t rsa -b 4096</code>	Create public/private key pair
Check port open	<code>netstat -tuln</code>	<code>netstat -tuln grep :22</code>	Is SSH port listening?

[BUILDING] REAL-WORLD EXAMPLE**AWS Production Deployment****Situation:**

- DevOps engineer needs to deploy code update to 10 production AWS EC2 servers

What they did:

- Stored deployment config as env vars: `export ENV=production DB_HOST=rds.aws.com`
- SSH into each server: `ssh -i ~/.ssh/aws-prod.pem ubuntu@server-ip`
- Script reads ENV variable to load correct config — zero hardcoded passwords in code
- Automated with Ansible: run across all 10 servers simultaneously

Result:

- Zero passwords stored in code — env vars provide secrets securely
- Same deployment script works in Dev, Staging, Production — just different ENV value
- SSH key authentication: no passwords to guess, full audit trail, revoke access by removing key

[!] COMMON MISCONCEPTIONS**[X] Myth:**

Closing the terminal loses environment variables — and that is a bug

[OK] Reality:

This is EXPECTED behaviour. Non-exported variables last only for the current session. Add permanent vars to `~/.bashrc` or use a `.env` file with tools like `direnv` for project-specific variables

[X] Myth:

Sharing your SSH private key (`.pem`) so others can access the server

[OK] Reality:

NEVER share your private key. Create a new key pair for each person: `ssh-keygen` → copy THEIR public key to `authorized_keys`. To revoke access: remove their public key. Shared private key = shared secret = security breach

[>] TRY IT YOURSELF

1. Set an env var: `export MY_NAME="YourName"` → `echo $MY_NAME` → close terminal → reopen → `echo $MY_NAME` (notice it is gone) → add it to `~/.bashrc` to make it permanent
2. Generate SSH keys: `ssh-keygen -t rsa -b 4096` → press Enter for all prompts → `cat ~/.ssh/id_rsa.pub` to see your public key
3. If you have an AWS free tier server: `ssh -i your-key.pem ubuntu@your-ec2-ip` → once in, run `env` to see all environment variables → exit with `logout`

[OK] QUICK SUMMARY

- **Env vars:** key=value pairs programs read for config — export makes them visible to child processes
- **Permanent vars:** add `export VAR=value` to `~/.bashrc` — loads on every terminal open
- **IP + Port:** IP = building address (find the machine), Port = office number (find the service)
- **Key ports:** 22=SSH, 80=HTTP, 443=HTTPS, 3306=MySQL, 5432=PostgreSQL
- **SSH = encrypted tunnel:** controls any Linux server remotely — foundation of all DevOps remote work
- **SSH keys:** public key on server, private key on your laptop — NEVER share private key

[?] MINI QUIZ — Test Your Understanding

Q1. Why store `DB_PASSWORD` as an environment variable instead of hardcoding it in application code?

- a) Env vars are faster to read than hardcoded values
- b) Hardcoded passwords get committed to Git repos — anyone with repo access gets the password
- c) Env vars are encrypted automatically
- d) There is no real reason — it is just convention

[OK] Answer: b — Hardcoded secrets in code end up in Git history forever. Env vars keep secrets out of code — changed per environment without code changes

Q2. You connect to an AWS server with: `ssh -i mykey.pem ubuntu@54.1.2.3`. What is the significance of port 22?

- a) Port 22 is the HTTP web port
- b) Port 22 is the default SSH port — SSH listens here unless changed
- c) Port 22 is the database port
- d) Port 22 is chosen randomly per connection

[OK] Answer: b — SSH uses port 22 by default. The server has an SSH daemon listening on port 22. You can change it in `/etc/ssh/sshd_config` for security hardening

Q3. What is the CRITICAL rule about SSH key pairs?

- a) Share both keys freely — they are just random numbers
- b) Private key stays on your machine only — public key can be shared
- c) Public key must be kept secret
- d) Use the same key pair for all servers

[OK] Answer: b — Private key = your secret identity. Public key = the lock. Share the lock freely (put it on servers), keep the key on your machine. Anyone with your private key can impersonate you

Glossary — Day 02: Linux for DevOps

apt	Advanced Package Tool — Ubuntu/Debian package manager: <code>sudo apt install [package]</code>
Bash	Bourne Again Shell — default command interpreter on most Linux systems
.bashrc	Hidden config file in home dir — runs on every new terminal — store permanent env vars here
cgroups	Linux kernel feature — limits/isolates resources per process (basis for containers)
chmod	Change file permissions — <code>chmod 755</code> (numeric) or <code>chmod +x</code> (symbolic)
chown	Change file owner — <code>chown user:group filename</code>
Cron	Task scheduler — runs commands/scripts automatically at specified times
crontab	Cron table — file listing scheduled cron jobs. <code>crontab -e</code> to edit
/dev	Device directory — all hardware (disks, terminals) represented as files here
DNS	Domain Name System — translates <code>google.com</code> to <code>142.250.80.46</code>
/etc	System configuration files — <code>nginx.conf</code> , <code>ssh/sshd_config</code> , <code>hosts</code> , <code>fstab</code>
Environment Variable	Named key=value setting read by programs — <code>export VAR=value</code>
execute (x)	File permission allowing a file to be run as a program
export	Makes a variable available to child processes/programs in the session
/home	User personal directories — <code>/home/username</code> is your home folder
Host OS	The real physical OS — runs the Hypervisor in Type 2 virtualization
Guest OS	OS running inside a Virtual Machine — thinks it has real hardware
grep	Global Regular Expression Print — searches text for patterns
Hypervisor	Software that creates and manages Virtual Machines (VMs)
Kernel	Core of the OS — only component that directly communicates with hardware
ln -s	Create symbolic link (shortcut/alias) to a file or directory
localhost	<code>127.0.0.1</code> — always refers to the current machine
Pipe ()	Passes stdout of one command as stdin of the next command
PATH	Env var listing directories Linux searches for executable programs
Port	Number identifying a specific service on a machine — <code>22=SSH</code> , <code>80=HTTP</code>
pwd	Print Working Directory — shows your current location in the file system
read (r)	File permission allowing a file to be viewed/read

Repository	Official online library of packages for a package manager
root	(1) The / directory — base of the entire file system. (2) The superuser account
Root (/)	Starting point of the Linux file system — every path begins here
rwX	Permission notation: r=read(4), w=write(2), x=execute(1)
Shebang	<code>#!/bin/bash</code> — first line of script declaring the interpreter to use
Shell	User interface to OS — interprets commands, sends them to Kernel
Snapshot	Point-in-time VM state backup — restore to undo any changes
SSH	Secure Shell — encrypted protocol for remote Linux server access
ssh-keygen	Command to generate SSH key pair (private + public key)
sudo	SuperUser DO — runs one command with root privileges (logged)
Tilde (~)	Shortcut for current user home directory — <code>cd ~</code> goes home
Type 1 Hypervisor	Bare-metal Hypervisor — runs directly on hardware (KVM, VMware ESXi)
Type 2 Hypervisor	Hosted Hypervisor — runs as app on Host OS (VirtualBox, VMware Workstation)
uname -r	Prints current Linux kernel version
/var/log	Directory containing system and application log files
VM (Virtual Machine)	Isolated virtual computer running inside a physical computer
write (w)	File permission allowing a file to be modified or deleted
755	Numeric permission: owner rwx, group r-x, others r-x — standard for executables
644	Numeric permission: owner rw-, group r--, others r-- — standard for config files

Resources & Further Learning — Module 2

Essential Reads

- Linux Command Line by William Shotts — free at linuxcommand.org
- "The Linux Command Line" (O'Reilly) — best book for CLI mastery
- Linux Foundation Training — lf-training.io — official courses
- Ryan's Tutorials — ryantutorials.net/linuxtutorial — highly beginner-friendly

Practice Platforms (All Free)

- Killercoda (killercoda.com) — browser Linux terminal, zero install
- OverTheWire: Bandit — security-focused Linux challenges, wargames
- Linux Journey — linuxjourney.com — interactive lessons with exercises
- TryHackMe Linux rooms — free browser VMs for Linux practice

YouTube (Best Channels)

- NetworkChuck — Linux for beginners, energetic style, practical labs
- TechWorld with Nana — Linux for DevOps series, Kubernetes context
- The Linux Experiment — in-depth Linux explanations, open source
- TutorialLinux — deep bash scripting and system administration

Hands-On Labs

- WSL2 (Windows): `wsl --install` in PowerShell — Linux on Windows in 5 min
- VirtualBox + Ubuntu ISO — local VM for safe experimentation
- AWS free tier EC2 — real cloud Linux server, 750 hrs/month free
- Play with Docker — labs.play-with-docker.com — Alpine Linux in browser

Certifications

- Linux Foundation Certified SysAdmin (LFCS) — \$395 — hands-on exam
- CompTIA Linux+ — \$358 — vendor-neutral Linux certification
- Red Hat Certified System Admin (RHCSA) — enterprise standard
- AWS Solutions Architect — covers Linux on cloud (SAA-C03)

Community & Repo

- <https://github.com/aryashreep/learn-devops-in-90-days> — this series repo
- [r/linux4noobs](#) and [r/linuxadmin](#) — questions answered fast
- Unix & Linux Stack Exchange — unix.stackexchange.com — expert Q&A;
- Linux Foundation Slack — slack.linuxfoundation.org — community

Start Today: Open Your Terminal and Type: `uname -r`

Day 02 Complete [OK] • Next: Day 03 — Git & Version Control

<https://github.com/aryashreep/learn-devops-in-90-days> • DevOps 90-Day Training 2026 • Aryashree Pritikrishna